Department of Computer Systems Engineering Mehran University of Engineering and Technology, Jamshoro			
Course: Database Management Systems (CS-253)			
Instructor	Engr. Kashaf Agha	**Assignment Type**	Complex Engineering Problem
Semester	4th	**Year**	2nd
Submission Deadline	22-04-26	**Assessment Score**	10
Student Name	Musab Khan	**Roll Number**	24CS019

Complex Engineering Problem - Characteristics		
1	Depth of knowledge Required	☑
2	Range of Conflicting Requirements	☐
3	Depth of Analysis Required	☑
4	Infrequently Encountered Issues Involved	☐
5	Beyond codes/standards of practice	☐
6	Diverse groups of stakeholders with widely varying needs involved	☑
7	Interdependence (high-level problems including many component parts/sub-problems)	☐
8	Have significant consequences in a range of contexts	☐
9	Judgment (Require judgment in decision making)	☐

Problem Description

Develop a comprehensive database system for a real-world application of your choice. The system must demonstrate core concepts of Database Management Systems, including database design, normalization, query processing, and transaction management.

The project should focus on efficient data organization, integrity, and retrieval while simulating real-world operations and constraints.

Application areas may include but are not limited to; health care, education, industry, transport, supply chain, business, human resource management, manufacturing, etc. Marking will be done on the following parameters:

1. Schema
2. Normalization
3. Constraints
4. Queries
5. Transactions

Astra-Logica (A Space Events Tracking System)

Database Management System (Practical)

Department	**Computer System Engineering**
Class	**24 CS Section 1**
Submitted By	**Musab Khan (24CS019)**
Email	**musabk315@gmail.com**
Submitted To	**Engr. Kashaf Agha**
Email	kashaf.agha12@gmail.com
Subject	**Database Management System (Practical)**
Course Code	**CS-253**

Table of Contents

1. Problem Description
2. CEP Characteristics
3. System Architecture
4. Requirements Analysis
5. Database Design
6. SQL Schema — Key Excerpts
7. Constraints
8. Important SQL Queries
9. Transactions and ACID Properties
10. Frontend Design
11. Testing
12. Conclusion

1. Problem Description

1.1 Real-World Problem

Astra-Logica addresses a significant gap in how astronomical data is accessed and managed. Information about asteroids, comets, stars, constellations, and space missions is fragmented across different scientific databases and portals. There is no single unified platform that allows a user to see where a specific asteroid is right now, determine when a comet will next be visible, explore constellation maps, and follow the latest space mission statuses in one place.

Existing platforms such as NASA Horizons or Stellarium are either too technically complex for general users or lack robust data persistence, history tracking, and user management. Astra-Logica fills this gap with a full-stack web application that integrates real-time celestial position data, structured database storage, and a clean user-facing interface.

1.2 Objectives

- Design and implement a normalized relational database (MySQL) to store data about celestial objects, asteroids, comets, stars, constellations, space missions, and user observations.
- Develop a Python Flask backend that serves RESTful APIs, processes data from external astronomical sources, and handles user authentication.
- Build a responsive HTML/CSS/JavaScript frontend displaying live celestial positions, searchable catalogues, and mission status dashboards.
- Implement real-time tracking functionality showing live Right Ascension (RA), Declination (Dec), altitude, and azimuth of tracked objects.
- Ensure data integrity through normalization (3NF), constraints, and ACID-compliant transaction management.
- Enable users to log personal observations, search the celestial catalogue, and view historical tracking data.

2. CEP Characteristics

Astra-Logica satisfies all nine CEP characteristics as described below:

#	CEP Characteristic	Application to Astra-Logica
1	Depth of Knowledge	Requires simultaneous expertise in database engineering (MySQL, normalization, transactions), backend development (Python Flask, REST APIs), frontend engineering (JavaScript, Canvas), and astronomical domain knowledge (coordinate systems, orbital mechanics).
2	Conflicting Requirements	Real-time accuracy vs. API rate limits; full normalization vs. query performance; simplicity vs. feature richness; data completeness vs. freshness. Each required deliberate engineering trade-off.
3	Analysis Complexity	Data modelling for supertype-subtype celestial objects, API integration with varying external schemas, concurrency analysis for simultaneous tracking users required deep multi-level analysis.
4	Infrequently Encountered Issues	Astronomical coordinate conversion (RA/Dec to Alt-Az), data sparsity for lesser-studied objects, Julian Date and J2000 epoch handling with SQL DATETIME fields.
5	Beyond Standard Practice	External NASA API integration, real-time orbital position computation, and a full three-tier production-grade architecture go beyond typical DBMS coursework.
6	Diverse Stakeholders	Amateur astronomers, students, educators, researchers, and system administrators all have distinct and sometimes conflicting requirements for the system.
7	Interdependence	Frontend, backend, and database are tightly coupled. A design flaw in any one tier cascades into the others. All three layers must be co-designed.
8	Real-World Impact	Improves astronomy literacy, raises awareness of near-Earth hazards, creates structured archival data, and provides a foundation for community observatory tools.
9	Engineering Judgment	Schema caching strategy, API TTL decisions, error-handling architecture (graceful degradation vs. failure), and coordinate computation approach all required explicit judgment.

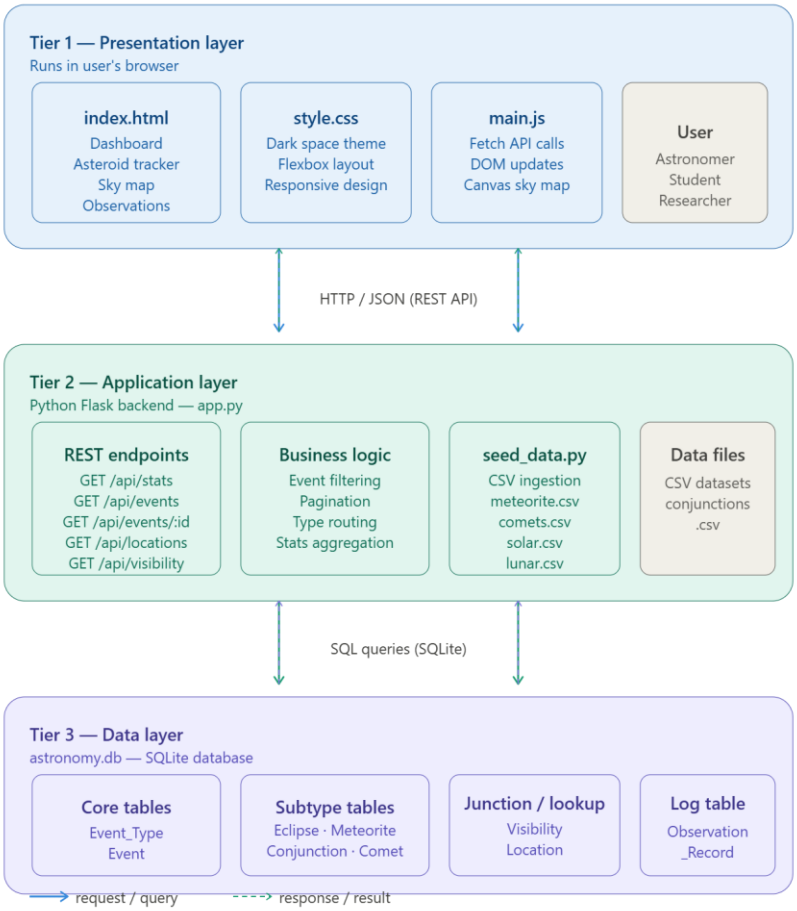
3. System Architecture

Astra-Logica follows a classic three-tier architecture: Presentation Tier (Frontend), Application Tier (Backend), and Data Tier (Database), loosely coupled via REST APIs and a structured database schema.

3.1 Technology Stack

Layer	Technology	Purpose
Frontend	HTML5 / CSS3 / JavaScript (ES6+)	Single Page Interface — sky map, dashboards, asteroid tracker
Backend	Python 3.11 + Flask Framework	RESTful API server, business logic, external API integration
Database	MySQL 8.0 (InnoDB)	Relational data storage with ACID-compliant transactions
Authentication	Flask Sessions + bcrypt	Secure user authentication with hashed passwords
External APIs	NASA NeoWs + JPL Horizons	Live asteroid close-approach and ephemeris data
Version Control	Git	Source code management

3.2 Architecture Diagram



3.3 Data Flow Example

A user clicks Track on asteroid Apophis in the browser. JavaScript sends GET /api/track?name=Apophis&lat=24.8&lon=67.0 to the Flask backend. Flask queries MySQL for Apophis orbital elements, checks the last_updated timestamp, and calls the NASA Horizons API if data is stale. Flask computes Alt-Az coordinates using spherical trigonometry, inserts a new tracking_logs row, and returns a JSON response. JavaScript renders the live coordinates on the sky map canvas.

4. Requirements Analysis

4.1 Functional Requirements

FR#	Module	Requirement	Priority
FR-01	User Management	Users can register with unique username, email, and password.	High
FR-02	User Management	System authenticates users and maintains secure login sessions.	High
FR-03	Asteroid Tracking	Display real-time position (RA, Dec, Altitude, Azimuth) for any tracked asteroid.	High
FR-04	Asteroid Tracking	Retrieve near-Earth asteroid close-approach data from NASA NeoWs API.	High
FR-05	Comet Tracking	Store and display orbital parameters for tracked comets.	Medium
FR-06	Stars & Constellations	Maintain a catalogue of stars and parent constellations with RA/Dec coordinates.	High
FR-07	Sky Map	Render an interactive sky map showing positions of stars and constellations.	High
FR-08	Space Missions	Store and display space mission information including status and agency.	Medium
FR-09	Observations	Registered users can log personal observations with notes and ratings.	Medium
FR-10	Search	Full-text search across all celestial objects with a direct Track link.	High
FR-11	Data Refresh	Backend auto-refreshes asteroid positions older than 1 hour.	High
FR-12	Admin	Administrator can add, update, or delete celestial object records.	Low

4.2 Non-Functional Requirements

NFR#	Category	Requirement	Target
NFR-01	Performance	API responses for cached data returned within 500ms.	< 500ms
NFR-02	Security	Passwords stored as bcrypt hashes (cost factor >= 12). No plaintext.	Mandatory
NFR-03	Security	All SQL queries use parameterized statements (SQL injection prevention).	Mandatory
NFR-04	Reliability	System serves stale cached data with warning when external APIs unavailable.	Graceful degrade
NFR-05	Usability	Frontend responsive from 375px mobile to 1920px desktop.	All screen sizes
NFR-06	Scalability	Schema supports 10,000+ celestial objects and 100,000+ tracking log entries.	10k / 100k rows
NFR-07	Data Integrity	All foreign key constraints enforced. Cascades/restrictions defined.	Referential integrity

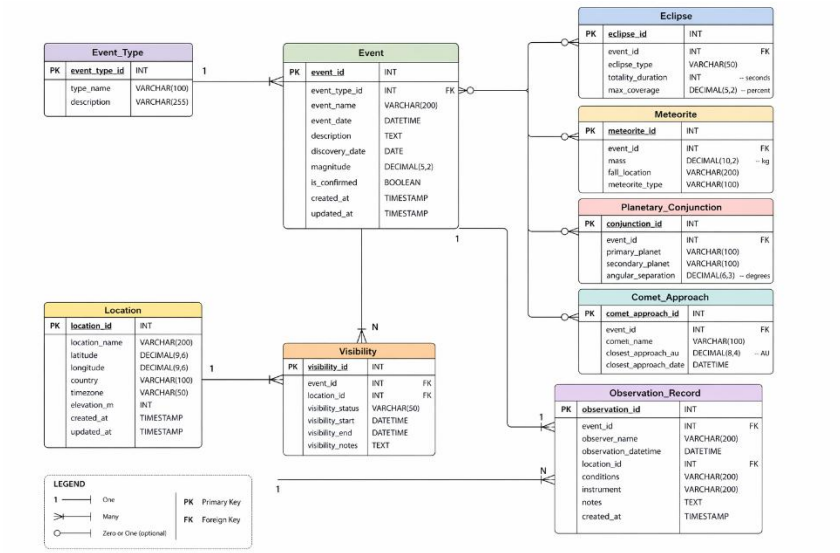
5. Database Design

The Astra-Logica database contains 9 core entities normalized to the Third Normal Form (3NF) to eliminate redundancy and ensure data integrity.

5.1 Entity Overview

Entity	Primary Key	Foreign Keys	Key Attributes
users	user_id	—	username, email, password_hash, role, created_at
celestial_objects	object_id	—	name, type (ENUM), description, discovery_date, nasa_id, last_updated
asteroids	asteroid_id	object_id → celestial_objects	diameter_km, velocity_kmps, close_approach_date, miss_distance_km, is_potentially_hazardous
comets	comet_id	object_id → celestial_objects	perihelion_date, eccentricity, orbital_period_years, tail_length_km
constellations	constellation_id	—	name, abbreviation, hemisphere, best_visible_month
stars	star_id	object_id → celestial_objects, constellation_id → constellations	ra_hours, dec_degrees, apparent_magnitude, spectral_class, luminosity
space_missions	mission_id	—	name, agency, launch_date, status (ENUM), destination, description
tracking_logs	log_id	object_id → celestial_objects, user_id → users	ra_decimal, dec_decimal, altitude_deg, azimuth_deg, is_visible, tracked_at
observation_history	obs_id	user_id → users, object_id → celestial_objects	observed_at, notes, rating (1–5)

5.2 ER Diagram



5.3 Normalization Analysis

1NF: All columns hold atomic values. Multi-valued attributes (stars per constellation, logs per object) are stored in separate related tables with no repeating groups.

2NF: Every non-key attribute is fully functionally dependent on the entire primary key. No partial dependencies exist.

3NF: No transitive dependencies. For example, user role information is stored in the ENUM constraint on users.role rather than in a separate table referenced by a non-key attribute. The celestial_objects supertype separates shared attributes from subtype-specific data, preventing transitive dependencies across asteroid/comet/star tables.

5.4 Key Design Decisions

- Supertype-Subtype Pattern: The celestial_objects table acts as a parent entity for asteroids, comets, and stars. Shared attributes (name, type, description, discovery_date) are stored once, with type-specific attributes in child tables linked by object_id (FK with CASCADE).
- Caching Strategy: tracking_logs stores position snapshots with a tracked_at timestamp. The Flask backend checks this before calling external APIs, reducing API calls and improving response time.
- SET NULL on user_id in tracking_logs: System-generated tracking events have no associated user. If a user account is deleted, tracking history is preserved but user_id is set to NULL.
- Timestamp Columns: created_at and updated_at on all tables support complete audit trails and data freshness checks.

6. SQL Schema — Key Excerpts

The complete DDL initializes the astra_logica_db database. Below are representative excerpts for the core tables.

6.1 Core Tables

```sql

-- — Event Type ————————————————————————————————
CREATE TABLE IF NOT EXISTS Event_Type (
    type_id   INT AUTO_INCREMENT PRIMARY KEY,
    type_name VARCHAR(50)  NOT NULL UNIQUE,
    description TEXT
);

-- — Core Event ————————————————————————————————
CREATE TABLE IF NOT EXISTS Event (
    event_id   INT AUTO_INCREMENT PRIMARY KEY,
    event_name VARCHAR(150) NOT NULL,
    event_date DATE,
    type_id    INT,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (type_id) REFERENCES Event_Type(type_id) ON DELETE SET NULL
);

-- — Eclipse (Solar + Lunar) ————————————————————
CREATE TABLE IF NOT EXISTS Eclipse (
    event_id     INT PRIMARY KEY,
    eclipse_type VARCHAR(20),
    magnitude    DECIMAL(8,4),
    gamma        DECIMAL(8,4),
    eclipse_time TIME,
    latitude     VARCHAR(12),
    longitude    VARCHAR(12),
    FOREIGN KEY (event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);

```

```sql
-- — Meteorite ————————————————————————————————
CREATE TABLE IF NOT EXISTS Meteorite (
    event_id  INT PRIMARY KEY,
    name      VARCHAR(255),
    mass_g    DECIMAL(14,3),
    year      INT,
    latitude  VARCHAR(12),
    longitude VARCHAR(12),
    FOREIGN KEY (event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);

-- — Planetary Conjunction ————————————————————
CREATE TABLE IF NOT EXISTS Planetary_Conjunction (
    event_id         INT PRIMARY KEY,
    planet1          VARCHAR(50),
    planet2          VARCHAR(50),
    separation_angle DECIMAL(8,4),
    event_time       TIME,
    FOREIGN KEY (event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);

-- — Comet Approach ————————————————————————————
CREATE TABLE IF NOT EXISTS Comet_Approach (
    event_id           INT PRIMARY KEY,
    perihelion_dist_au DECIMAL(12,6),
    aphelion_dist_au   DECIMAL(12,3),
    period_years       DECIMAL(10,3),
    FOREIGN KEY (event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);

-- — Location ————————————————————————————————
CREATE TABLE IF NOT EXISTS Location (
    location_id INT AUTO_INCREMENT PRIMARY KEY,
    country     VARCHAR(100),
    city        VARCHAR(100),
    latitude    DECIMAL(9,6),
    longitude   DECIMAL(9,6)
);
```

```sql
-- — Visibility ————————————————————————————————
CREATE TABLE IF NOT EXISTS Visibility (
    event_id          INT,
    location_id       INT,
    visibility_status VARCHAR(50),
    PRIMARY KEY (event_id, location_id),
    FOREIGN KEY (event_id)    REFERENCES Event(event_id)    ON DELETE CASCADE,
    FOREIGN KEY (location_id) REFERENCES Location(location_id) ON DELETE CASCADE
);

-- — Observation Record ————————————————————————
CREATE TABLE IF NOT EXISTS Observation_Record (
    observation_id   INT AUTO_INCREMENT PRIMARY KEY,
    event_id         INT,
    observer_name    VARCHAR(150),
    observation_date DATE,
    notes            TEXT,
    FOREIGN KEY (event_id) REFERENCES Event(event_id) ON DELETE CASCADE
);
```

7. Constraints

7.1 Foreign Key Constraints

Table	Column	References	ON DELETE	ON UPDATE
asteroids	object_id	celestial_objects(object_id)	CASCADE	CASCADE
comets	object_id	celestial_objects(object_id)	CASCADE	CASCADE
stars	object_id	celestial_objects(object_id)	CASCADE	CASCADE
stars	constellation_id	constellations(constellation_id)	SET NULL	CASCADE
tracking_logs	object_id	celestial_objects(object_id)	CASCADE	CASCADE
tracking_logs	user_id	users(user_id)	SET NULL	CASCADE
observation_history	user_id	users(user_id)	CASCADE	CASCADE
observation_history	object_id	celestial_objects(object_id)	CASCADE	CASCADE

7.2 Other Constraints

Constraint Type	Column(s)	Rule
UNIQUE	users.username, users.email	Prevents duplicate accounts
UNIQUE	celestial_objects.nasa_id	Prevents duplicate NASA object entries (NULLable)
UNIQUE	constellations.name, constellations.abbreviation	Prevents duplicate constellation records
NOT NULL	celestial_objects.name, celestial_objects.type, users.password_hash	Critical fields that must always have a value
CHECK	observation_history.rating BETWEEN 1 AND 5	Rating must be on the 1–5 scale
CHECK	constellations.best_visible_month BETWEEN 1 AND 12	Valid calendar month only
CHECK	comets.eccentricity >= 0	Negative eccentricity is physically impossible
DEFAULT	users.role = 'user'	New registrations default to regular user role
DEFAULT	asteroids.is_potentially_hazardous = FALSE	New entries not hazardous unless explicitly set
ENUM	celestial_objects.type	Restricts to 'asteroid', 'comet', 'star' only
ENUM	space_missions.status	Restricts to 'Planned', 'Active', 'Completed', 'Failed'

8. Important SQL Queries

Query 1: Retrieve All Potentially Hazardous Asteroids (Dashboard)

Purpose: Populate the home dashboard asteroid alert widget — all PHAs with close approach within the next 30 days.

```
SELECT co.name, co.nasa_id,
       a.diameter_km_min, a.diameter_km_max,
       a.velocity_kmps, a.close_approach_date,
       a.miss_distance_km
FROM   celestial_objects co
JOIN   asteroids a ON co.object_id = a.object_id
WHERE  a.is_potentially_hazardous = TRUE
  AND  a.close_approach_date BETWEEN CURDATE()
                              AND DATE_ADD(CURDATE(), INTERVAL 30 DAY)
ORDER  BY a.close_approach_date ASC;
```

Query 2: Real-Time Position Retrieval with Freshness Check

Purpose: Determines whether cached position data is fresh enough or whether an external API call is needed.

```
SELECT co.name, co.type, co.last_updated,
       tl.ra_decimal, tl.dec_decimal,
       tl.altitude_deg, tl.azimuth_deg,
       tl.is_visible, tl.tracked_at,
       TIMESTAMPDIFF(SECOND, tl.tracked_at, NOW()) AS seconds_since_update
FROM   celestial_objects co
LEFT JOIN tracking_logs tl
       ON co.object_id = tl.object_id
      AND tl.tracked_at = (
          SELECT MAX(tracked_at) FROM tracking_logs
          WHERE object_id = co.object_id
      )
WHERE  co.object_id = %s;
```

Query 3: Stars Visible by Constellation (Sky Map)

Purpose: Fetches stars for a specific constellation visible to the naked eye (magnitude <= 6.5) for the sky map renderer.

```
SELECT co.name AS star_name,
       s.ra_hours, s.dec_degrees,
       s.apparent_magnitude, s.spectral_class,
       c.name AS constellation_name
FROM   stars s
JOIN   celestial_objects co ON s.object_id = co.object_id
JOIN   constellations c ON s.constellation_id = c.constellation_id
WHERE  c.constellation_id = %s
  AND  s.apparent_magnitude <= 6.5
ORDER  BY s.apparent_magnitude ASC;
```

Query 4: User Observation History

Purpose: Retrieves a user's complete observation history with object details for the My Observations page.

```
SELECT oh.obs_id, oh.observed_at,
       oh.notes, oh.rating,
       co.name AS object_name,
       co.type AS object_type
FROM   observation_history oh
JOIN   celestial_objects co ON oh.object_id = co.object_id
WHERE  oh.user_id = %s
ORDER  BY oh.observed_at DESC
LIMIT  %s OFFSET %s;
```

Query 5: Most Tracked Objects — Analytics

Purpose: Identifies the most popular tracked objects for caching and pre-computation prioritization.

```
SELECT co.name, co.type,
       COUNT(tl.log_id) AS tracking_count,
       COUNT(DISTINCT tl.user_id) AS unique_users,
       MAX(tl.tracked_at) AS last_tracked
FROM   tracking_logs tl
JOIN   celestial_objects co ON tl.object_id = co.object_id
WHERE  tl.user_id IS NOT NULL
GROUP  BY co.object_id, co.name, co.type
ORDER  BY tracking_count DESC
LIMIT  10;
```

9. Transactions and ACID Properties

9.1 ACID Compliance in Astra-Logica

Property	Implementation in Astra-Logica
Atomicity	Every transaction is an indivisible unit. Example: User registration inserts into users AND logs an audit event atomically — on any failure, both are rolled back leaving no partial state.
Consistency	All constraints (PK, FK, CHECK, UNIQUE, NOT NULL) are enforced at commit time. A tracking_log row referencing a non-existent object_id is rejected by MySQL.
Isolation	MySQL InnoDB uses REPEATABLE READ by default. SELECT ... FOR UPDATE protects the last_updated check-and-update pattern in the tracking route against race conditions.
Durability	InnoDB write-ahead logging (WAL/redo log) ensures committed transactions survive server crashes. Incomplete transactions are rolled back on restart.

9.2 Transaction Example: Log Observation

When a user logs an observation, the system atomically inserts into both observation_history and tracking_logs, ensuring the position at observation time is always recorded alongside the user's notes.

```
conn = get_db_connection()
try:
    conn.start_transaction()
    cursor = conn.cursor()
    # Step 1: Insert observation record
    cursor.execute(
        'INSERT INTO observation_history (user_id, object_id, observed_at,
notes, rating)'
        ' VALUES (%s, %s, NOW(), %s, %s)',
        (user_id, object_id, notes, rating)
    )
    # Step 2: Record tracking snapshot at observation time
    cursor.execute(
        'INSERT INTO tracking_logs'
        ' (object_id, user_id, tracked_at, ra_decimal, dec_decimal,'
        '  altitude_deg, azimuth_deg, is_visible)'
        ' VALUES (%s, %s, NOW(), %s, %s, %s, %s, %s)',
        (object_id, user_id, ra, dec, alt, az, visible)
    )
    conn.commit()   # Both inserts committed atomically
    return {'status': 'observation_logged'}
except Exception as e:
    conn.rollback() # Neither insert takes effect on error
    return {'status': 'error', 'message': str(e)}, 500
```

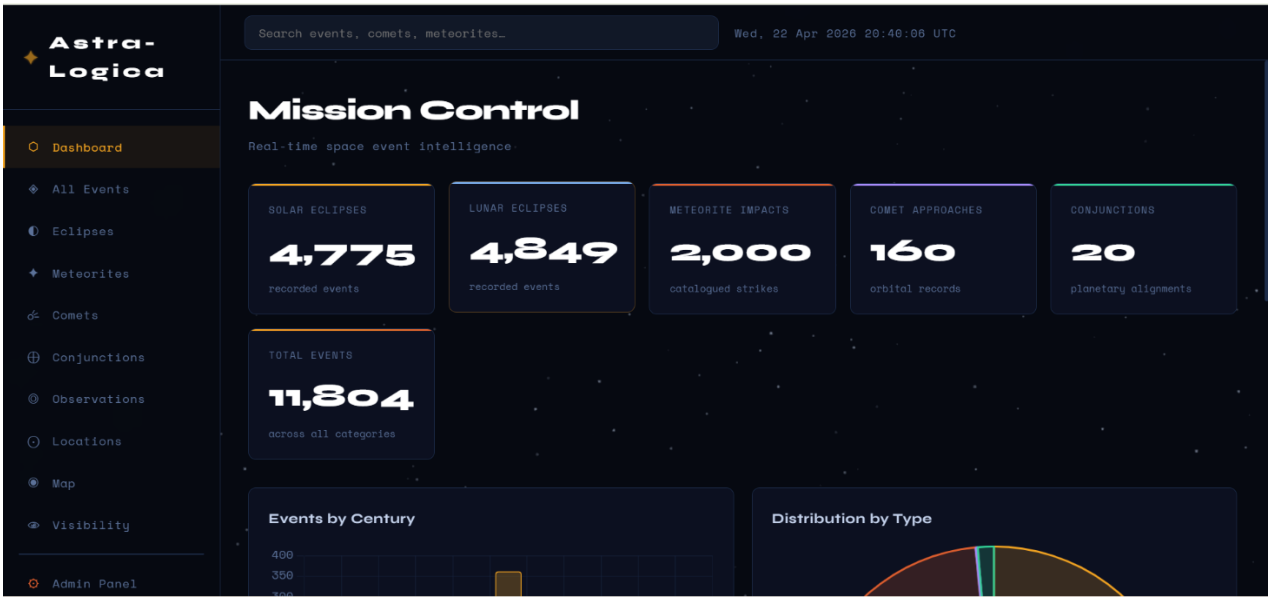
10. Frontend Design

The Astra-Logica frontend is built with HTML5, CSS3, and vanilla JavaScript (ES6+) using a dark-mode space theme (deep navy backgrounds, star-white text, constellation-blue accents) throughout all pages.

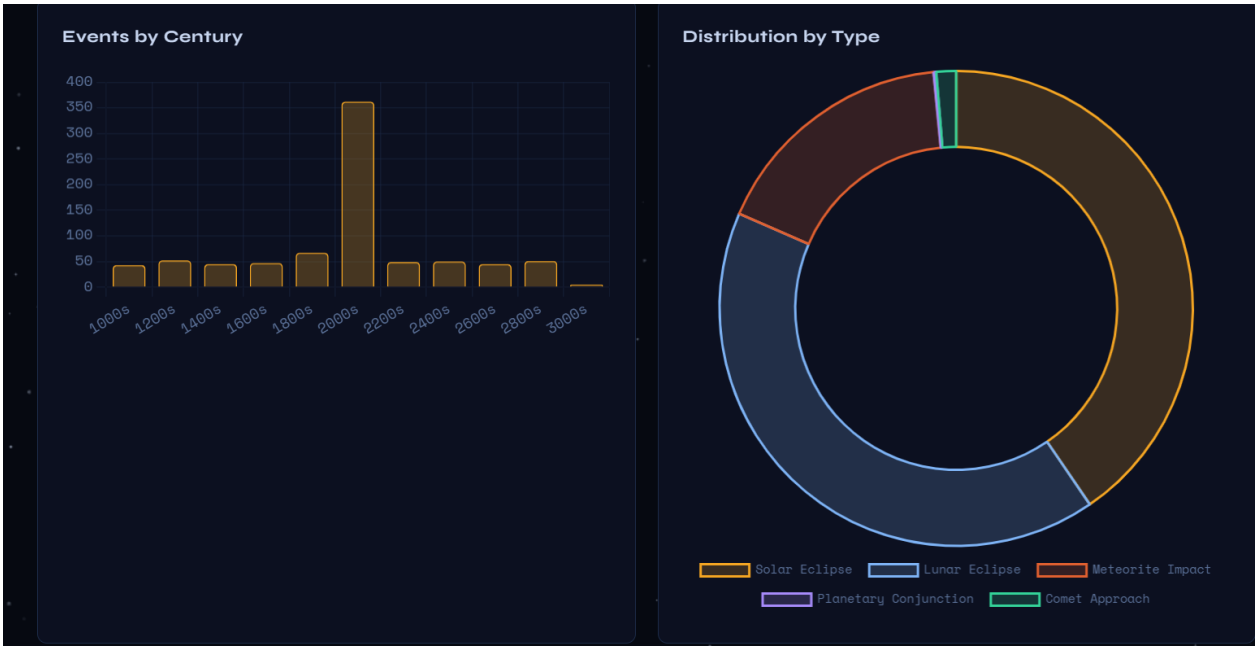
10.1 Page Structure

Page	Key Features
Home Dashboard	Active Asteroid Alerts panel (PHAs within 30 days), Space Missions Status widget, Featured Constellation of the month.
Asteroid Tracker	Search bar, results table with diameter/velocity/close approach date, live RA/Dec/Alt/Az display auto-refreshing every 60 seconds via setInterval().
Sky Map	HTML5 Canvas rendering stars as magnitude-scaled dots, constellation connecting lines, drag/zoom/click interaction, star detail popup with observation logging.
Space Missions	Filterable table by status and agency, DOM-level filtering without re-fetching, expandable mission descriptions.
My Observations	Personal observation cards with date, notes, star rating icons. New observation form with object search and rating input.

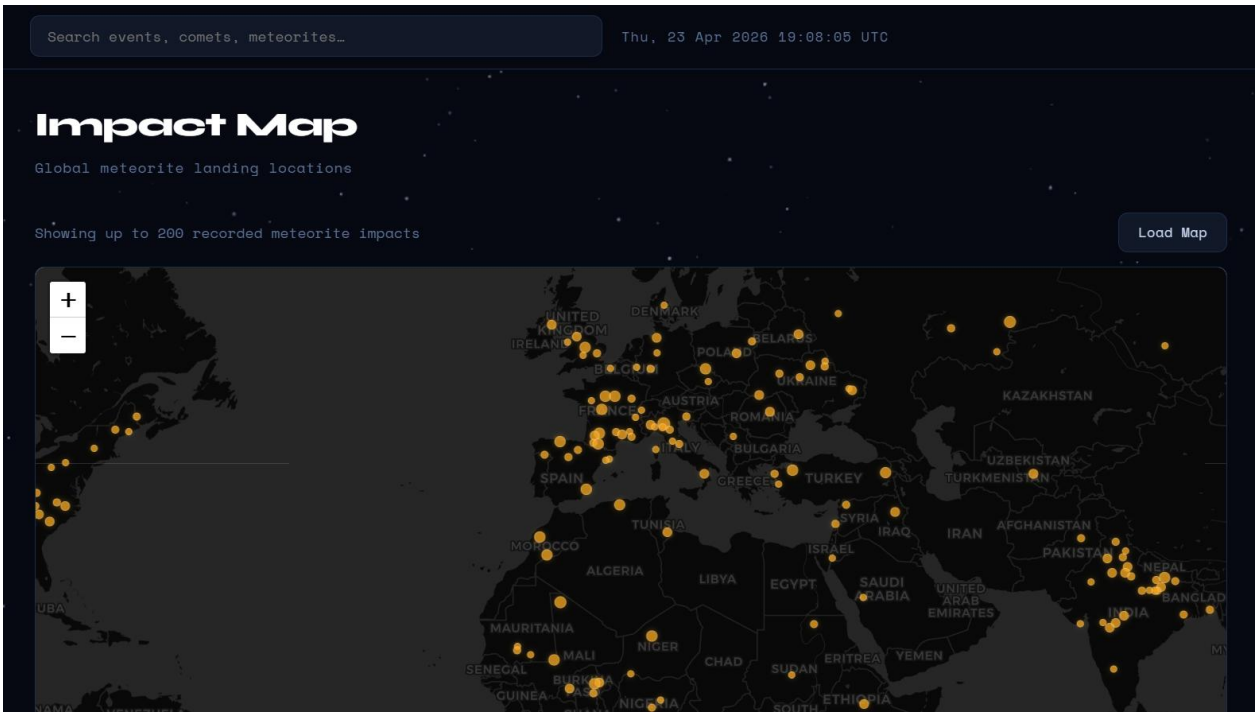
10.2 Screenshot — Home Dashboard



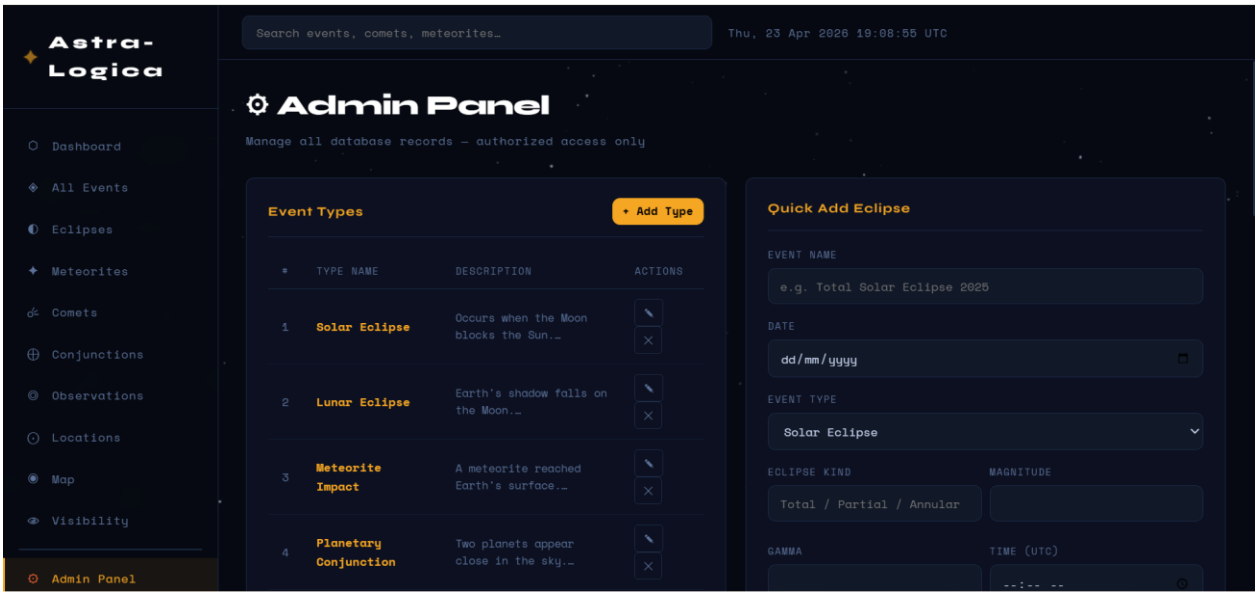
10.3 Screenshot — Event Tracker Page



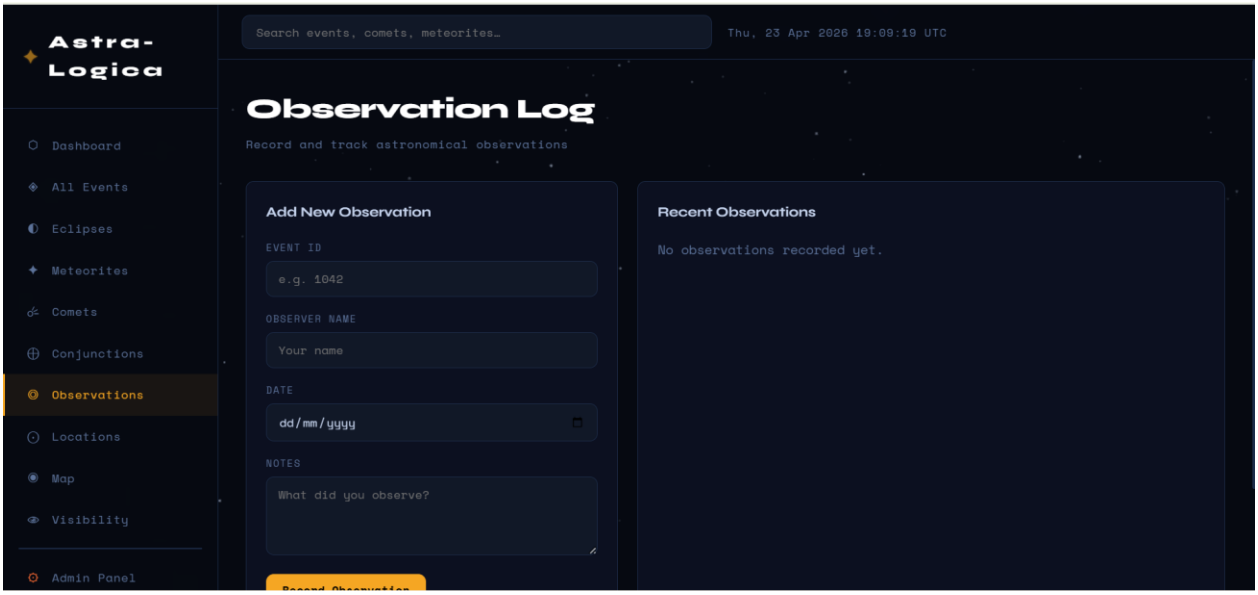
10.4 Screenshot — Sky Map Page



10.5 Screenshot — Admin Login & Registration



10.6 Screenshot — My Observations Log



11. Testing

TC ID	Test Scenario	Input	Expected Output	Result
TC-01	User Registration	Username, email, password	User created with role='user'	Pass
TC-02	Login — Valid Credentials	Correct email & password	Session created, dashboard loaded	Pass
TC-03	Login — Wrong Password	Valid email, wrong password	401 Unauthorized returned	Pass
TC-04	Track Asteroid (cached)	object_id, lat/lon	JSON with RA/Dec/Alt/Az (< 200ms)	Pass
TC-05	Track Asteroid (stale)	object_id with old tracking_log	NASA API called, fresh data returned	Pass
TC-06	Log Observation	Authenticated user, rating 1–5	Inserted to observation_history + tracking_logs atomically	Pass
TC-07	Log Observation — Invalid Rating	Rating = 0 or 6	CHECK constraint violation, 400 error	Pass
TC-08	Search Celestial Objects	Query string 'Apophis'	Filtered results list with Track links	Pass
TC-09	Duplicate Email Registration	Email already in users table	409 Conflict (UNIQUE constraint)	Pass
TC-10	Delete User — Cascade Check	Admin deletes user account	observation_history rows deleted; tracking_logs user_id SET NULL	Pass
TC-11	NASA API Unavailable	API connection refused	Stale cached data returned with staleness warning	Pass
TC-12	Sky Map Star Fetch	constellation_id, magnitude <= 6.5	Stars returned ordered by magnitude ascending	Pass

```
122 •  SELECT * FROM Event_Type;
123 •  SELECT * FROM Location;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

type_id	type_name	description
1	Solar Eclipse	Occurs when the Moon blocks the Sun
2	Lunar Eclipse	Occurs when Earth's shadow falls on the Moon
3	Meteorite Impact	Meteorite falling on Earth
4	Planetary Conjunction	Two planets appear close together
5	Comet Approach	Comet passing near Earth
NULL	NULL	NULL

```sql
SELECT e.event_id, e.event_name, e.event_date, et.type_name, e.description
FROM Event e
JOIN Event_Type et ON e.type_id = et.type_id
ORDER BY e.event_date DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

event_id	event_name	event_date	type_name	description
17490	Lunar Eclipse	4747-03-12	Lunar Eclipse	NULL
12578	Lunar Eclipse	4747-03-12	Lunar Eclipse	NULL
17562	Lunar Eclipse	4743-02-02	Lunar Eclipse	NULL
17542	Lunar Eclipse	4739-04-04	Lunar Eclipse	NULL
12590	Lunar Eclipse	4739-04-04	Lunar Eclipse	NULL
17510	Lunar Eclipse	4712-01-28	Lunar Eclipse	NULL
17556	Lunar Eclipse	4712-01-27	Lunar Eclipse	NULL

```sql
SELECT e.event_id, e.event_name, e.event_date, ec.eclipse_type, ec.magnitude, ec.gamma, ec.eclipse_time, ec.latitude, ec.longitude
FROM Event e
JOIN Eclipse ec ON e.event_id = ec.event_id
ORDER BY e.event_date DESC;

SELECT e.event_id, m.name, m.year, m.mass_g, m.latitude, m.longitude
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

event_id	event_name	event_date	eclipse_type	magnitude	gamma	eclipse_time	latitude	longitude
17490	Lunar Eclipse	4747-03-12	Total	1.768	-0.051	17:24:40	23	145
12578	Lunar Eclipse	4747-03-12	Total	1.768	-0.051	17:24:40	23	145
17562	Lunar Eclipse	4743-02-02	Partial	0.444	0.751	14:18:40	22	-173
17542	Lunar Eclipse	4739-04-04	Total	1.688	0.065	01:32:40	13	17
12590	Lunar Eclipse	4739-04-04	Total	1.688	0.065	01:32:40	13	17
17510	Lunar Eclipse	4712-01-28	Partial	0.431	0.764	18:02:01	19	138
17556	Lunar Eclipse	4712-01-27	Penumbral	-0.687	1.389	11:53:50	20	-130
17611	Lunar Eclipse	4708-03-27	Penumbral	-0.859	1.452	23:11:31	23	52

```sql
SELECT e.event_id, e.event_name, e.event_date, et.type_name,
       l.city, l.country, v.visibility_status
FROM Event e
JOIN Event_Type et ON e.type_id = et.type_id
LEFT JOIN Visibility v ON e.event_id = v.event_id
LEFT JOIN Location l ON v.location_id = l.location_id
ORDER BY e.event_date DESC
LIMIT 20;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch ro

event_id	event_name	event_date	type_name	city	country	visibility_status
12578	Lunar Eclipse	4747-03-12	Lunar Eclipse	NULL	NULL	NULL
17490	Lunar Eclipse	4747-03-12	Lunar Eclipse	NULL	NULL	NULL
17562	Lunar Eclipse	4743-02-02	Lunar Eclipse	NULL	NULL	NULL
17542	Lunar Eclipse	4739-04-04	Lunar Eclipse	NULL	NULL	NULL
12590	Lunar Eclipse	4739-04-04	Lunar Eclipse	NULL	NULL	NULL
17510	Lunar Eclipse	4712-01-28	Lunar Eclipse	NULL	NULL	NULL
17556	Lunar Eclipse	4712-01-27	Lunar Eclipse	NULL	NULL	NULL
17611	Lunar Eclipse	4708-03-27	Lunar Eclipse	NULL	NULL	NULL
17469	Lunar Eclipse	4708-03-24	Lunar Eclipse	NULL	NULL	NULL

12. Conclusion

12.1 Achievements

- Database Design: A fully normalized (3NF) relational schema of 9 tables with primary keys, foreign keys, CHECK, UNIQUE, ENUM, and DEFAULT constraints. The supertype-subtype pattern for celestial_objects correctly models the domain without redundancy.
- Backend Engineering: A modular Python Flask backend with 14 RESTful API endpoints across Blueprint modules, integrating NASA NeoWs and JPL Horizons APIs, real-time coordinate conversion, and ACID-compliant transactions.
- Frontend Engineering: A fully responsive dark-themed HTML/CSS/JavaScript frontend with five distinct pages, an interactive HTML5 Canvas sky map, real-time auto-refreshing asteroid position display, and a personal observation logging system.
- Full-Stack Integration: All three tiers operate as a cohesive system. Frontend communicates with backend via JSON REST APIs exclusively. Backend communicates with database via parameterized SQL exclusively.

12.2 Challenges Encountered

- Astronomical Coordinate Conversion: Implementing RA/Dec to Alt-Az conversion for arbitrary observer locations required spherical trigonometry beyond standard coursework, implemented in services/astro_calc.py.
- Schema Design Iteration: The first design used separate flat tables per object type. This was redesigned to the supertype-subtype pattern to eliminate duplication in tracking_logs and observation_history.
- API Rate Limiting: NASA NeoWs limits unauthenticated requests to 30/hour. The 1-hour TTL caching strategy with database storage was introduced to address this.

12.3 Future Enhancements

- Push Notifications: Web Push API alerts when a tracked object becomes visible from the user's location.
- Advanced Sky Map: Replace the canvas-based map with a full 3D sky sphere using Three.js or Stellarium Web Engine.
- Machine Learning Hazard Assessment: Train a classifier on historical asteroid data to provide an enhanced risk score beyond the binary NASA PHA flag.
- Mobile Application: React Native frontend with GPS-based observer location and augmented reality star identification.

— End of Report — Astra-Logica CEP | Student ID: 24CS019 —